Text File Handling in Java

19 August 2026 09:43

Text file handling means creating, opening, reading, writing, appending, modifying and deleting files that contain data in the form of characters or human-readable text.

Java provides several classes for text - file handling through packages such as: java.io and in modern Java: ***java.nio.file***

For beginners, the java.io classes such as FileReader, FileWriter, BufferedReader and BufferedWriter are particularly useful

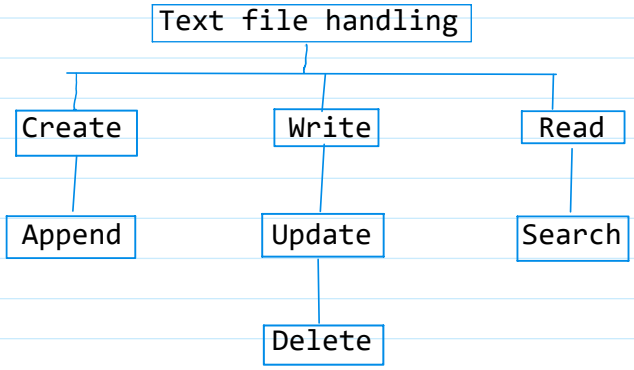
What is a Text file?
A text file stores information as characters that can normally be viewed using a text editor.

Examples:
- Student.txt
- Employees.txt
- Marks.txt
- Data.csv

For example, student.txt might contain:

101, Ajay, 87
102, Anchit, 90
103, Faisal, 78

Text file handling Operations
A Java program can perform several operations on text files:



Common operations are:
i. Create a file.
ii. Write data.
iii. Read the data.
iv. Append data.
v. Search data.
vi. Update data.
vii. Delete data.

viii. **Rename a file.**
ix. **Check file properties.**

Important Java Classes

File: Represents a file or directory.
It can be used to:
i. Check whether a file exist.
ii. Create a file.
iii. Delete a file.
iv. Get file name.
v. Get file size.
vi. Check whether it is a directory.

FileReader: Used to read character data from a text file.

FileWriter: Used to write character data to a text file.

BufferedReader: Reads text efficiently, particularly line by line.

BufferedWriter: Writes text efficiently.

PrintWriter: Provides convenient methods such as:
print()
println()
printf()
For writing text

Writing to what text file using FileWwriter

```
package FileHandling;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class WtiteDataInFile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true)){
            fw.write("101, Rahul, 85\n");
            fw.write("102, Anchit, 95\n");
            fw.write("103, Aaditya, 55\n");
            fw.write("104, Raman, 65\n");
            fw.write("105, Aman, 75\n");
            System.out.println("Data written successfully.");
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
```

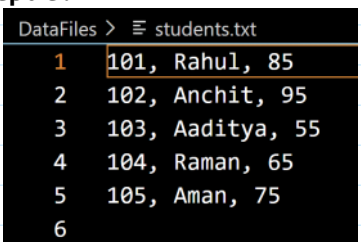
'append' option is activated.

This is called try-with-resource.

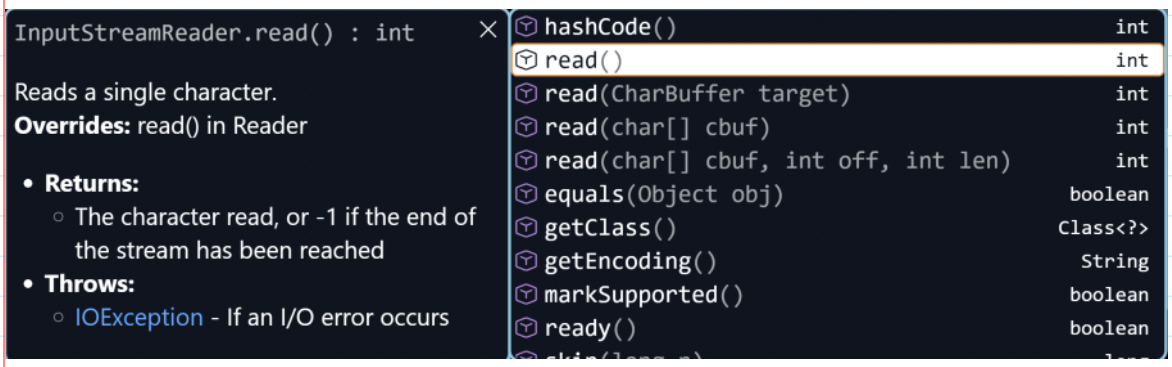
↑ Java automatically closes fw when the try block finishes.
It is generally preferable to manual calling close().

```
            sc.close();
        }
    }
}
```

Output:



Reading a text file using FileReader

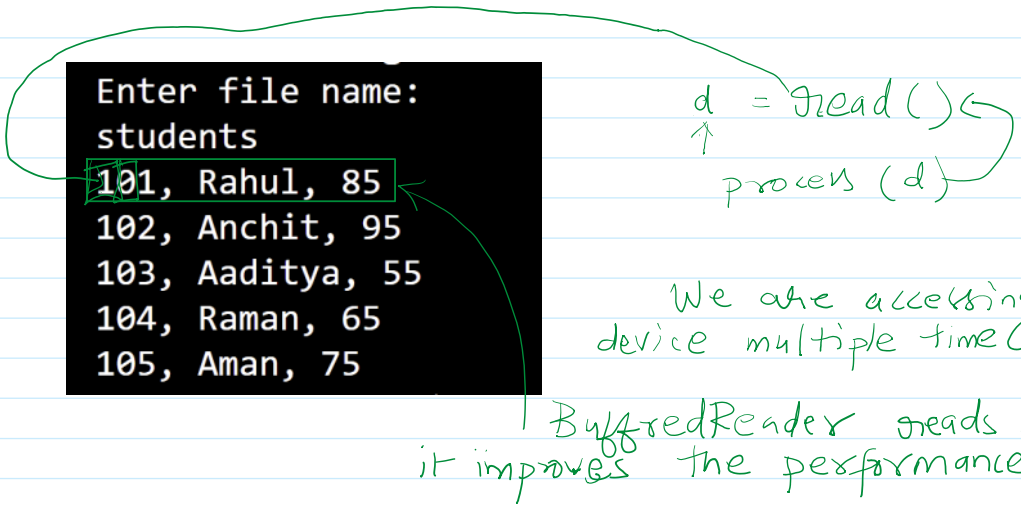



```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadDataFromTextfile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileReader fr = new FileReader(path)){
            int d;
            while((d = fr.read()) != -1){
                System.out.print((char)d);
            }
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
```

```
            sc.close();
        }
    }
}
```

Output:

```
Enter file name:
students
101, Rahul, 85
102, Anchit, 95
103, Aaditya, 55
104, Raman, 65
105, Aman, 75
```




```
BufferedReader.readLine() : String
```

Reads a line of text. A line is considered to be terminated by any one of a line feed ('\n'), a carriage return ('\r'), a carriage return followed immediately by a line feed, or by reaching the end-of-file (EOF).

- **Returns:**
  - A String containing the contents of the line, not including any line-termination characters, or null if the end of the stream has been reached without reading any characters
- **Throws:**
  - IOException - If an I/O error occurs
- **See Also:**
  - java.nio.file.Files.readAllLines

```
package FileHandling;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadingFileUsingBufferedReader {
```

```
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileReader fr = new FileReader(path);
    BufferedReader br = new BufferedReader(fr)){
            String line;
            while((line = br.readLine()) != null){
                System.out.println(line);
            }
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:

```
Enter file name:
students
101, Rahul, 85
102, Anchit, 95
103, Aaditya, 55
104, Raman, 65
105, Aman, 75
```

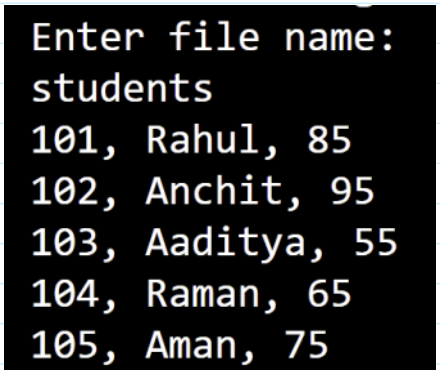
Reading a file into a String

We can use Files.readString() in modern Java.

```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Scanner;
public class ReadingFileInAString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try{
            String data = Files.readString(Path.of(path));
```

```
            System.out.println(data);
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:



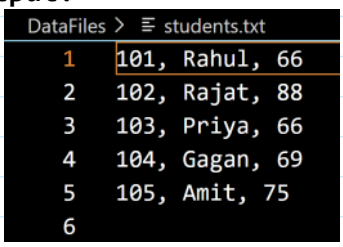
Writing Using BufferedWriter

Code:

```
package FileHandling;
import java.io.BufferedWriter;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class BufferedWriterDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true);
    BufferedWriter bw = new BufferedWriter(fw)){
            do{
                System.out.println("Enter roll number: ");
                int rn = sc.nextInt();
                sc.nextLine();
                System.out.println("Enter name: ");
                String name = sc.nextLine();
                System.out.println("Enter marks: ");
                int marks = sc.nextInt();
                sc.nextLine();
```

```
                    bw.write(rn+", "+name+", "+marks);
                    bw.newLine();
                    System.out.println("Press Y/y to store more records: ");
                    String ch = sc.next();
                    if(ch.equalsIgnoreCase("y")){
                        continue;
                    }
                    else{
                        break;
                    }
                }while(true);
            }
            catch(FileNotFoundException fnfe){
                fnfe.printStackTrace();
            }
            catch(IOException e){
                e.printStackTrace();
            }
            finally{
                sc.close();
            }
        }
    }
```

Output:



Using PrintWriter: Printer Writer is a convenient when writing formatted text.

Code:

```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;
public class PrintWriterDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true);
    PrintWriter pw = new PrintWriter(fw)){
            do{
                System.out.println("Enter roll number: ");
```

```
                int rn = sc.nextInt();
                sc.nextLine();
                System.out.println("Enter name: ");
                String name = sc.nextLine();
                System.out.println("Enter marks: ");
                int marks = sc.nextInt();
                sc.nextLine();
                // pw.println(rn+", "+name+", "+marks);
                pw.printf("%d, %s, %d%n", rn, name, marks);
                System.out.println("Press Y/y to store more records: ");
                String ch = sc.next();
                if(ch.equalsIgnoreCase("y")){
                    continue;
                }
                else{
                    break;
                }
            }while(true);
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:

DataFiles > students.txt

```
1   101, Rahul, 66
2   102, Rajat, 88
3   103, Priya, 66
4   104, Gagan, 69
5   105, Amit, 75
6   106, Jiya, 99
7   107, Anchit, 100
```

WAP to perform a search operation by taking roll_number as input. If found display the record, otherwise print record not found.